Jobsheet Praktikum: Pertemuan 4

Relasi Kelas dan Array Objek

Mata Kuliah	Praktikum Pemrograman Berbasis Objek (RTI253008)
Pertemuan	4 (Minggu 4)
Durasi	1 × 4 × 50’ praktikum; 1 × 1 × 50’ tugas/laporan mandiri

A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, mahasiswa mampu:

1. Menulis kelas yang memiliki kelas lain sebagai atributnya (relasi association).
2. Mengelola sekumpulan objek lewat array yang menjadi atribut sebuah kelas (relasi aggregation), termasuk pencarian di dalamnya.

B. Persiapan dan Prasyarat

- **Alat**: JDK 17 atau lebih baru, NetBeans (editor yang digunakan sepanjang praktikum ini).
- **Proyek**: pertemuan ini melanjutkan proyek `bank-mini` dari Pertemuan 3.

> **Tanpa NetBeans?** Jobsheet ini tetap dapat diikuti menggunakan editor teks biasa:
>
> ```
> javac -d out src/id/ac/polinema/*.java
> java -cp out id.ac.polinema.Main
> ```
>
> Checkpoint dan output yang dihasilkan tetap sama persis, apa pun editor yang digunakan.

C. Langkah Kerja

Langkah 1: Account Berelasi dengan Customer

Proyek `bank-mini` dari Pertemuan 3 dilanjutkan pada pertemuan ini. Sejauh ini, `Account` hanya menyimpan nama pemiliknya sebagai teks biasa. Tambahkan kelas `Customer`:

```
1  package id.ac.polinema;
2
3  public class Customer {
4      private String name;
5      private String phone;
6
7      public Customer(String name, String phone) {
8          this.name = name;
9          this.phone = phone;
10     }
11
12     public String getName() {
13         return name;
14     }
15
16     public String getPhone() {
17         return phone;
18     }
19 }
```

Figure 1: Customer.java

Ganti isi `Account.java` sehingga menyimpan referensi ke sebuah `Customer` (association), bukan lagi nama pemilik berupa teks:

Karena constructor `Account` kini menerima objek `Customer`, perbarui pengujian di `Main.java`:

```java
package id.ac.polinema;

public class Account {
    private String accountNumber;
    private Customer owner;
    private double balance;

    public Account(String accountNumber, Customer owner, double balance) {
        this.accountNumber = accountNumber;
        this.owner = owner;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public Customer getOwner() {
        return owner;
    }

    public double getBalance() {
        return balance;
    }

    public boolean deposit(double amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }

    public boolean withdraw(double amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    public void printInfo() {
        System.out.println(accountNumber + " - " + owner.getName() + " - balance: " + balance);
    }
}
```

Figure 2: Account.java, atribut ownerName diganti menjadi owner bertipe Customer

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        Account acc1 = new Account("A001", customer1, 500000);
        acc1.withdraw(150000);
        acc1.printInfo();
    }
}
```

Figure 3: Main.java memakai Customer dan constructor Account yang baru

> ✅ **Checkpoint:** program berhasil dikompilasi ulang dan menampilkan `A001 - Nadia - balance: 350000.0`, kali ini diambil melalui objek `Customer`.

> ⚠ **Jika gagal:** apabila muncul galat `incompatible types: String cannot be converted to Customer`, periksa apakah argumen kedua pada `new Account(...)` sudah berupa objek `Customer`, bukan teks nama pemilik.

Langkah 2: Bank Mengelola Banyak Account

Tambahkan kelas Bank, yang menyimpan banyak `Account` dalam sebuah array (aggregation):

```java
package id.ac.polinema;

public class Bank {
    private Account[] accounts;
    private int count;

    public Bank(int capacity) {
        accounts = new Account[capacity];
        count = 0;
    }

    public boolean addAccount(Account account) {
        if (count >= accounts.length) {
            return false;
        }
        accounts[count] = account;
        count++;
        return true;
    }

    public Account findAccount(String accountNumber) {
        for (int i = 0; i < count; i++) {
            if (accounts[i].getAccountNumber().equals(accountNumber)) {
                return accounts[i];
            }
        }
        return null;
    }

    public void printAllAccounts() {
        for (int i = 0; i < count; i++) {
            accounts[i].printInfo();
        }
    }
}
```

Figure 4: Bank.java

Perbarui `Main.java` agar membuat dua rekening dan mengelolanya lewat Bank:

```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        Account acc1 = new Account("A001", customer1, 500000);
        acc1.withdraw(150000);

        Customer customer2 = new Customer("Sari", "0812-0000-0002");
        Account acc2 = new Account("A002", customer2, 200000);

        Bank bank = new Bank(10);
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.printAllAccounts();

        Account found = bank.findAccount("A002");
        if (found != null) {
            found.printInfo();
        }
    }
}
```

Figure 5: Main.java memakai Bank untuk mengelola dua Account

> ✅ **Checkpoint:** program mencetak dua baris dari `printAllAccounts()` (`A001 - Nadia - balance: 350000.0` dan `A002 - Sari - balance: 200000.0`), diikuti satu baris lagi hasil

`findAccount("A002")` yang menampilkan data yang sama untuk `A002`.

> ⚠ **Jika gagal:** apabila `findAccount(...)` selalu mengembalikan `null` walaupun nomor rekeningnya ada, periksa apakah perbandingan menggunakan `.equals(...)`, bukan ==, karena == pada `String` membandingkan referensi, bukan isi teksnya.

D. Tugas dan Hasil Kerja

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program setelah Langkah 2.
- **Tugas mandiri:**
  1. Seorang customer bisa saja memiliki lebih dari satu rekening. Tambahkan method `findAccountsByOwnerName(String name)` ke Bank, yang mengembalikan array berisi seluruh `Account` milik customer dengan nama tersebut:

```java
package id.ac.polinema;

public class Bank {
    private Account[] accounts;
    private int count;

    public Bank(int capacity) {
        accounts = new Account[capacity];
        count = 0;
    }

    public boolean addAccount(Account account) {
        if (count >= accounts.length) {
            return false;
        }
        accounts[count] = account;
        count++;
        return true;
    }

    public Account findAccount(String accountNumber) {
        for (int i = 0; i < count; i++) {
            if (accounts[i].getAccountNumber().equals(accountNumber)) {
                return accounts[i];
            }
        }
        return null;
    }

    public Account[] findAccountsByOwnerName(String name) {
        int matchCount = 0;
        for (int i = 0; i < count; i++) {
            if (accounts[i].getOwner().getName().equals(name)) {
                matchCount++;
            }
        }
        Account[] matches = new Account[matchCount];
        int index = 0;
        for (int i = 0; i < count; i++) {
            if (accounts[i].getOwner().getName().equals(name)) {
                matches[index] = accounts[i];
                index++;
            }
        }
        return matches;
    }

    public void printAllAccounts() {
        for (int i = 0; i < count; i++) {
            accounts[i].printInfo();
        }
    }
}
```

Figure 6: Bank.java dengan tambahan findAccountsByOwnerName

     Buktikan dengan membuat satu customer bernama “Nadia” yang memiliki dua `Account`, satu customer lain dengan satu `Account`, memasukkan ketiganya ke Bank, lalu memanggil `findAccountsByOwnerName("Nadia")` dan mencetak jumlah serta isi hasilnya.
  2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan):
     - (a) mengapa relasi `Account`-`Customer` disebut association, bukan aggregation atau composition?
     - (b) bayangkan objek Bank dihapus dari memori. Menurutmu, apakah objek `Account`

yang pernah ditambahkan ke dalamnya seharusnya ikut terhapus, atau tetap bisa berdiri sendiri? Apa maksud jawabanmu terhadap jenis relasi `Bank-Account`?

E. Kriteria Penilaian

Komponen	Bobot	Kriteria Lengkap (100%)	Kriteria Minimum
Langkah kerja tuntas	40%	Seluruh langkah dijalankan dan berfungsi	Sebagian besar langkah selesai, hasil akhir berjalan
Checkpoint terverifikasi	35%	Semua checkpoint tercapai dan dibuktikan (screenshot/output)	Sebagian checkpoint terbukti
Tugas mandiri	25%	Method pencarian per nama customer benar dan jawaban konsep tepat	Method pencarian ada meski jawaban belum lengkap